

JAVA 17 OCP · YDS STUDY SERIES

Unit 05 · Technical Memory Notes · Methods

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli ana nottaki](#) kuralları bir method tasarım ve çağrı algoritmasıyla tamamlar. Bir method sorusunda üç ayrı zamanı karıştırma: **declaration kontrolü** → **overload seçimi** → **runtime execution**. Dil çalışması: [vocabulary](#).

Kapsam denetimi

Ana çift dilli kaynak access, static context, effectively final, varargs, pass-by-value ve overload kurallarını tam kapsar. Bu hafıza notu şu bağları birlikte tekrar ettirir:

- method declaration parçaları ve signature tanımı,
- `protected` erişimin cross-package receiver kısıtı,
- overload applicability phase'leri,
- varargs'ın compile time'da array'e dönüşmesi,
- autoboxing/unboxing ile `null` runtime riski.

1. Method declaration anatomisi

```
public static final int count(String name, int... values)
    throws IllegalArgumentException {
    return values.length;
}
```

Declaration iskeleti: access modifier → optional specifiers → return type → method name → parameter list → optional `throws` list → body veya semicolon.

Bu bölümdeki non-generic method'larda signature, method name ve parameter type/order bilgisinden oluşur. Return type, access modifier, parameter names ve `throws` list signature'a dahil değildir. Generic method'larda type parameter'ları ve bunların uyarlanması da signature karşılaştırmasına katılır.

Bu yüzden yalnız return type'ı değiştirerek overload yapılamaz.

2. Local ve field modifiers

- Local variable access modifier veya `static` alamaz.
- Local variable `final` olabilir; `final var x = 3;` geçerlidir.
- Instance/static fields access modifier, `final`, `volatile`, `transient` gibi uygun modifier'lar olabilir.
- Blank instance `final` field, instance initializer veya constructor içinde atanabilir; normal tamamlanan her constructor yolu alanı tam bir kez initialize etmiş olmalıdır. Blank `static final` field, static initializer'da tam bir kez atanır.

`effectively final`, explicit `final` yazılmasa da initialization sonrasında reassign edilmeyen local/parameter demektir. Lambda/local class capture için gerekir.

3. Access matrix

Member access	Same class	Same package	Different package subclass	Other package non-subclass
<code>private</code>	Evet	Hayır	Hayır	Hayır
<code>package</code>	Evet	Evet	Hayır	Hayır
<code>protected</code>	Evet	Evet	Evet*	Hayır
<code>public</code>	Evet	Evet	Evet	Evet

`protected` yıldızı önemlidir: different package subclass, inherited member'a subclass context'i üzerinden erişir. Arbitrary parent reference üzerinden erişim verilmez.

```
package child;
class Cub extends parent.Lion {
    void test(Cub cub, parent.Lion lion) {
        cub.age = 2;    // age protected ise geçerli
        // lion.age = 3; // different package'ta geçersiz
    }
}
```

Hafıza cümlesi: Protected dış package'ta “subclass ol” demekle kalmaz; “subclass gözüyle bak” da der.

4. Static versus instance membership

- Instance method hem instance hem static member'a direct erişebilir.
- Static method direct olarak yalnız static member'a erişir.
- Static member class'a aittir; tüm instances tarafından paylaşılır.
- Static method'u instance expression üzerinden çağırmak derlenebilir, fakat method runtime object'e göre dispatch edilmez.

```
class Counter {
    static int total;
    int own;

    static void reset() {
        total = 0;
        // own = 0; // Does not compile: receiver yok
    }
}
```

Static initializer class initialization sırasında source order ile bir kez çalışır ve `this` kullanamaz.

5. Varargs: çağrıda esnek, içeride array

```
static int sum(String label, int... values) {
    return values.length;
}
```

- Bir method en fazla bir varargs parameter taşır.
- Varargs parameter son sırada olmalıdır.
- Method body içinde normal array gibi davranır.
- Caller sıfır, bir veya çok argument ya da doğrudan array verebilir.

```
sum("a");  
sum("b", 1, 2);  
sum("c", new int[] {1, 2});
```

Explicit `null` varargs array olarak geçirilebilir; body `values.length` çağırırsa `NullPointerException` oluşabilir.

6. Pass-by-value: kopyalanan şey variable value'sudur

Primitive için değerin, reference için reference value'nun kopyası aktarılır. Reference'ı nesneye giden bir bağlantı gibi düşün; Java kodu bunu ham bellek adresi olarak kullanmaz.

```
static void change(StringBuilder b) {  
    b.append("!");           // ortak object mutate edilir  
    b = new StringBuilder("new"); // yalnız local parameter reassign edilir  
}  
  
var x = new StringBuilder("old");  
change(x);  
System.out.println(x); // old!
```

Hafıza cümlesi: Java object'i değil, oku kopyalar; iki ok aynı object'i gösterebilir.

7. Overload resolution

Overload'lar same name, different parameter list taşır. Compiler call için applicable candidates'ı phase'ler hâlinde arar; daha erken phase'de uygun method bulursa sonraki phase'e geçmez.

Compiler üç applicability aşaması kullanır:

- 1 **Strict invocation:** Sabit argument sayısı ile identity veya widening conversion; boxing/unboxing yoktur.
- 2 **Loose invocation:** Sabit argument sayısı ile boxing/unboxing de kullanılabilir.
- 3 **Variable arity invocation:** Gerekirse ayrı argument'lar varargs array'inde toplanır.

İlk uygun aşamadaki adaylar arasından most specific method seçilir. Varargs bildirimi, çağrıya array verildiğinde ilk iki aşamada da fixed-arity method olarak değerlendirilebilir; “varargs method her zaman en sona kalır” genellemesi bu yüzden yanlıştır.

```
static void call(long x)    { System.out.print("long"); }  
static void call(Integer x) { System.out.print("Integer"); }  
static void call(int... x)  { System.out.print("varargs"); }  
  
call(3); // long: primitive widening, boxing'den önce
```

Önemli dönüşüm sınırları:

- Primitive widening sonra boxing yapılmaz: `int` doğrudan `Long` olamaz.
- Boxing sonra reference widening olabilir: `int` → `Integer` → `Object`.
- Birden fazla equally specific candidate kalırsa call ambiguous olur.
- Runtime object type yeni overload'ı görünür yapmaz; overload compile-time reference types ile seçilir.

8. Autoboxing, unboxing ve `null`

```
Integer boxed = 3; // boxing
int value = boxed; // unboxing
```

Wrapper `null` ise unboxing compile olur fakat runtime'da `NullPointerException` üretir.

```
Integer boxed = null;
// int value = boxed; // runtime NullPointerException
```

9. Return kontrolü

- `void` metod değer döndüremez. Non-void metod gövdesi sona düşerek normal tamamlanamaz; `return` varsa uygun türde değer ister. `throw` veya kesin sonsuz döngü nedeniyle normal tamamlanmayan bir gövde de derlenebilir.
- Return assignment conversion uygular; arbitrary narrowing otomatik yapılmaz.
- Returned reference yine pass-by-value'dur; object kopyalanmaz.

OCP için 20 saniyelik analiz sırası

- 1 Declaration, access ve static receiver birlikte geçerli mi?
- 2 Overload phase sırası doğru mu; varargs/ambiguity var mı?
- 3 Execution parameter reassign mı, object mutation mı yapıyor?
- 4 Boxing `null` riski ve bütün return path'leri kontrol edildi mi?

Aktif hatırlama · Özgün çalışma soruları

- 1 Return type method signature'ın parçası mıdır?
- 2 `int` argument için `long` ve `Integer` overload'ları varsa hangisi seçilir?
- 3 Reference parameter'a yeni object atamak caller variable'ını değiştirir mi?
- 4 Varargs parameter neden son sırada olmalıdır?

Cevaplar

- 1 Hayır; burada non-generic method için name ve parameter type/order esas alınır.
- 2 `long`; primitive widening boxing'den önce uygulanır.
- 3 Hayır; reference value'nun kopyası reassign edilir.
- 4 Önce gelseydi hangi argument'ların varargs'a, hangilerinin sonraki parameter'a ait olduğu belirlenemezdi.